

Taller 2 — RAG con Vector Search y Evaluación

Autor: Paolo A · **Curso:** IA Generativa, USFQ · **Fecha:** 2026-09-27

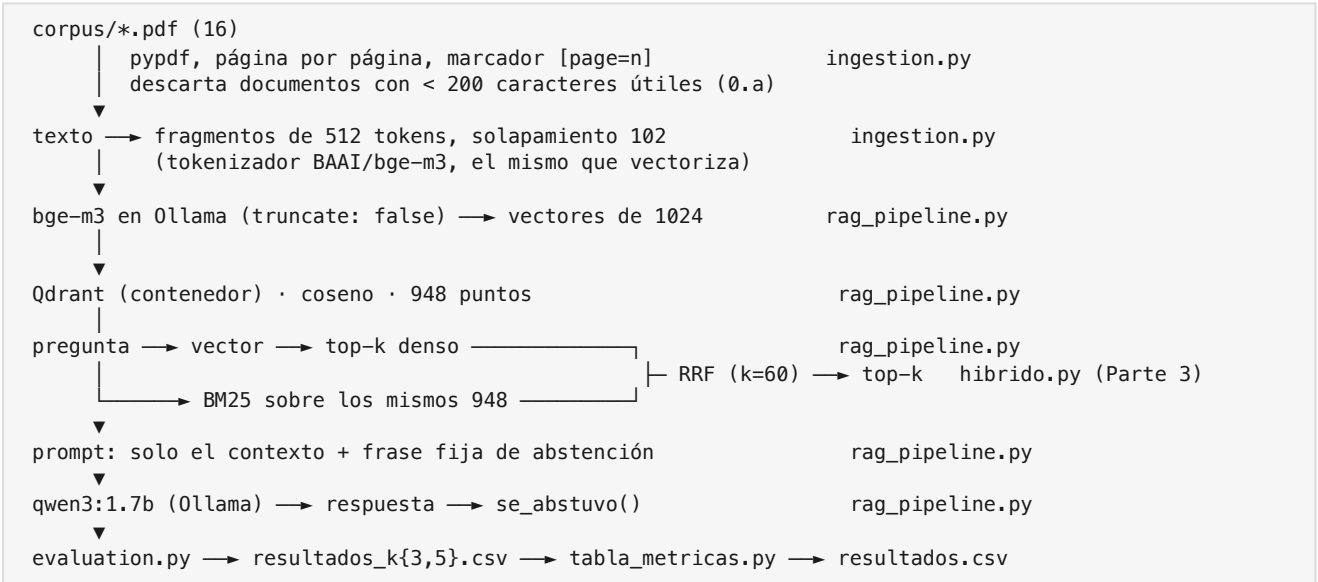
Repositorio: github.com/Pazzolo/Gen-Taller2

La regla del taller fue baseline → medir → extender → medir. Este informe junta las cinco partes. Todas las cifras salen de corridas reales, y cada tabla se puede reconstruir desde los CSV crudos del repositorio.

Resumen

Elemento	Detalle
Corpus	Opción B: 16 papers de las semanas 1 y 2, inglés , 440 páginas, todos con capa de texto
Fragmentación	tamaño fijo, 512 tokens con 102 de solapamiento , tokens de BAAI/bge-m3 → 948 fragmentos
Embeddings	fila embed_local_multilingue (BAAI/bge-m3, 1024 dim., 8192 tokens), verificada 2026-08-27 , servida por Ollama local (no la H200: no había VPN)
Índice	Qdrant 1.19.1 en contenedor, colección taller2_bge_m3_local , coseno
Generación	fila open_weight_pequeno (qwen3:1.7b) por Ollama local, temperatura 0
Golden set	10 preguntas: 4 simples, 3 multi-fragmento, 2 negativas, 1 adversarial (respondible)
Baseline	Hit Rate@3 0,875 , @5 1,000 · MRR 0,750 / 0,781 · abstención correcta 1,000 · indebida 0,250 / 0,125
Extensión	Opción B, híbrido BM25 + denso (RRF) : Hit Rate@3 1,000 · MRR 0,854 · indebida 0,125 / 0,000
Costo	USD 0 : ninguna llamada a API de pago

Arquitectura



Reproducibilidad

- **Versiones:** `requirements.txt` con las versiones exactas del entorno (Python 3.14.7).
- **Resultados crudos:** `resultados.csv` tiene las 20 filas del baseline (10 preguntas × k = 3 y 5). `tabla_metricas.py` reconstruye la tabla de la Parte 2.b desde ese archivo y comprueba que coincide con lo que reportó el evaluador.

- **Registro de corridas:** cada corrida escribe en `salidas/<modelo>@<host>/` y añade una fila a `experimentos.csv`. `EXPERIMENTOS.md` tiene el prefijo exacto de cada configuración y la tabla de todas las corridas, preparada para repetir el baseline en la H200 y comparar.
- **Credenciales:** ninguna clave en el repositorio. `.env` está en `.gitignore` y no está versionado, y una búsqueda del patrón de claves de OpenAI (`sk-...`) en los archivos versionados no encuentra nada.

Prefijo de todas las corridas del baseline:

```
export H200_EMBED_URL=http://localhost:11434 EMBEDDING_BACKEND=h200 OPENAI_API_KEY= \
GENERATION_MODEL=qwen3:1.7b QDRANT_COLLECTION=taller2_bge_m3_local
```

Presupuesto

No se gastó nada. Los embeddings (bge-m3) y la generación (qwen3:1.7b) corrieron en la Mac, por Ollama. Se intentó la ruta de OpenAI (embed_api_economico para embeddings y propietario_economico, gpt-4o-mini, para generar), pero la clave fue rechazada con un 401 antes de cualquier llamada facturable. Con esa ruta, el costo estimado de indexar habría sido de unos USD 0,01: 473 866 tokens a USD 0,02 por millón, precio de la fila verificado el 2026-08-27. RAGAS (Opción D) no se corrió, así que no hay estimación de llamadas de juez.

Tiempos en la Mac: indexar 948 fragmentos, 136,6 s; cada evaluación completa (10 preguntas, con generación), alrededor de 1,5 min por valor de k.

Parte 0 — Tres fallas que no fallan

Todo corre sobre `ejemplos/` con `EMBEDDING_BACKEND=local QDRANT_URL=":memory:"`, es decir, con el MiniLM multilingüe (`embed_notebook_s2`, tope de 128 tokens, verificado 2026-08-27), sin clave, sin VPN y sin Docker. No se modificó nada del andamiaje.

Para reproducir:

```
EMBEDDING_BACKEND=local QDRANT_URL=":memory:" CORPUS_DIR=ejemplos \  
python rag_pipeline.py "¿cuántos días de vacaciones puedo transferir?" # 0.a  
EMBEDDING_BACKEND=local QDRANT_URL=":memory:" python parte0.py # 0.a (contrafactual), 0.b,  
0.c
```

Las salidas crudas están en `salidas/parte0*.txt`.

0.a — El PDF que se indexa vacío

Salida cruda de la ingesta (`salidas/parte0a_ingesta.txt`, sin el prompt que imprime el modo inspección):

```
AVISO ingesta: instructivo_escaneado.pdf: 0 caracteres útiles en 2 página(s). ¿Escaneado o fotografiado?  
NO se indexa: sin texto no hay nada que recuperar (OCR aparte).  
embeddings: local · sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 · tope 128 tokens ·  
Qdrant :memory:  
ingesta: nimbus_gastos.md: 1 página(s), 294 caracteres útiles → 1 fragmento(s)  
ingesta: nimbus_remoto.md: 1 página(s), 296 caracteres útiles → 1 fragmento(s)  
ingesta: nimbus_vacaciones.md: 1 página(s), 363 caracteres útiles → 1 fragmento(s)  
indexados 3 fragmentos de 3 documento(s)
```

Lo que habría pasado sin la comprobación (`salidas/parte0a_sin_comprobacion.txt`):

```
documento: instructivo_escaneado.pdf · 2 página(s) · useful_chars=0 · parece_escaneado=True  
texto extraído por pypdf (repr): '\n[page=1]\n\n\n[page=2]\n'  
fragmentos que produciría la fragmentación sin la comprobación: 1  
[0] '[page=1] [page=2]' · caracteres útiles=0 (el filtro por fragmento exige 20)
```

`load_corpus` cuenta los caracteres alfanuméricos de cada documento sin contar los marcadores `[page=n]` que la propia ingesta añade, y si quedan menos de 200 (`MIN_CARACTERES_UTILES`) avisa y salta el documento: el instructivo escaneado tiene **0**. Sin esa comprobación, `pypdf` devuelve un texto vacío por página sin lanzar ninguna excepción, y la fragmentación convierte el documento en un único fragmento `'[page=1] [page=2]'` que no tiene ni una palabra del instructivo, como muestra el contrafactual de abajo.

Hay un detalle que se vio al correr el contrafactual: el andamiaje trae una segunda red, `MIN_CARACTERES_FRAGMENTO = 20`, que descarta fragmentos con menos de 20 caracteres útiles. Ese filtro también habría botado el `'[page=1] [page=2]'`, pero en silencio: el documento desaparecería sin aviso. La diferencia con la comprobación por documento es justamente el aviso. Con un PDF que tuviera, por ejemplo, una portada con texto y el resto escaneado, ninguno de los dos filtros saltaría, y el documento quedaría indexado a medias.

El corpus real no trae ningún documento así. `salidas/corpus_verificacion.txt` (rama `step-1-corpus`) muestra 16 documentos, todos indexables, y 0 páginas con menos de 50 caracteres útiles.

0.b — El fragmento que se corta a 128

Salida cruda (`salidas/parte0b_truncado.txt`):

aviso de la ingesta:

AVISO ingesta: chunk_tokens=900 supera el tope del modelo (128 tokens): lo que pase de 126 tokens no llega al índice –el MiniLM lo trunca en silencio; la H200 rechaza la petición–.

texto: salidas/parte0b_texto.txt (900 palabras)

tokens del texto (tokenizador del modelo, sin especiales): 1463

tope del modelo (max_seq_length): 128

fracción que llega al vector: 8.6% (126 tokens de contenido + 2 especiales)

coseno(vector del texto entero, vector del texto recortado a 128): 1.000000

El texto son las primeras 900 palabras que pypdf extrae de Vaswani et al. (2017). Esas 900 palabras son **1463 tokens** para el tokenizador del modelo, y el tope es **128**. El coseno entre el vector del texto entero y el del texto recortado es **1.000000**, o sea, son el mismo vector: el 91,4 % del texto no influye en nada de lo que se indexa. Un matiz: el tokenizador sí imprimió un aviso (1463 > 128) cuando se contaron los tokens, pero `encode` no dice nada. Quien solo llama a `encode`, que es lo que hace el pipeline, no se entera.

chunk_tokens para la Parte 1: 512, con overlap_tokens de 102 (un quinto). Que bge-m3 admita 8192 tokens solo dice que un fragmento de 8192 no se trunca; no dice que sea útil. Un fragmento así equivaldría a varias páginas de un paper mezclando secciones, y su vector sería un promedio de temas distintos que no se parecería mucho a ninguna pregunta concreta. Con la proporción que salió aquí, de 1463/900 $\approx$ 1,6 tokens por palabra (el tokenizador del MiniLM y el de bge-m3 vienen ambos de XLM-RoBERTa, así que la proporción debería ser parecida, aunque no se verificó), 512 tokens son unas 315 palabras: más o menos dos o tres párrafos de un paper, que es la escala a la que suele estar la respuesta de una pregunta simple. Este valor es provisional; en la Parte 1 se revisa contra la distribución real de largos de párrafo del corpus.

0.c — El índice que no se queja

Salida cruda (salidas/parte0c_indice_no_se_queja.txt):

pregunta: ¿cuál es la política de mascotas?

1. 0.058 nimbus_gastos.md / nimbus_gastos-0000 # Política de reembolso de gastos de NimbusSoft Los gastos de viaje se...

2. 0.032 nimbus_remoto.md / nimbus_remoto-0000 # Política de trabajo remoto de NimbusSoft El trabajo remoto está perm...

3. -0.033 nimbus_vacaciones.md / nimbus_vacaciones-0000 # Política de vacaciones de NimbusSoft Los empleados a tiempo completo...

pregunta: ¿qué exige el instructivo de prácticas de campo sobre el seguro de accidentes?

1. 0.229 nimbus_remoto.md / nimbus_remoto-0000 # Política de trabajo remoto de NimbusSoft El trabajo remoto está perm...

2. 0.171 nimbus_gastos.md / nimbus_gastos-0000 # Política de reembolso de gastos de NimbusSoft Los gastos de viaje se...

3. 0.099 nimbus_vacaciones.md / nimbus_vacaciones-0000 # Política de vacaciones de NimbusSoft Los empleados a tiempo completo...

Ninguna de las dos preguntas tiene respuesta en el índice, y el índice devuelve igual tres vecinos con puntaje, sin error ni aviso. El Hit Rate no puede detectar esto porque solo mide si el documento fuente aparece entre los k recuperados. En la primera pregunta no hay documento fuente; en la segunda, el documento fuente nunca entró al índice (0.a). En ninguno de los dos casos hay un «acierto» que pueda fallar. Por eso el golden set lleva preguntas negativas y se mide aparte si el sistema se abstiene.

Hay que ser honesto con lo que se ve aquí: los puntajes no son de «aspecto sano» en el sentido literal. Para la pregunta de mascotas el mejor es 0,058, y un umbral sobre el puntaje podría filtrarla. La segunda pregunta ya es más delicada: 0,229 es el puntaje más alto de esta corrida, y justamente es la pregunta cuya respuesta existe en un documento que no se indexó. Con un corpus de 3 fragmentos cortos cualquier umbral sería arbitrario, y el pipeline no tiene ninguno: pase lo que pase, entrega los k vecinos. En el corpus real conviene revisar si los puntajes de las negativas se separan o no de los de las respondibles; esa columna está en `resultados.csv (score_top1)`.

Parte 1 — Baseline RAG

Corpus: los 16 papers seminales de las semanas 1 y 2 (Opción B), **en inglés**, 440 páginas, todos con capa de texto (salidas/corpus_verificacion.txt).

Embeddings: **bge-m3**, fila `embed_local_multilingue`, servido por un **Ollama local** y no por el de la H200, porque no se tenía la VPN (el detalle está en 1.3). Generación: **qwen3:1.7b** por Ollama, fila `open_weight_pequeno`. Todo corre a costo cero. La configuración completa y la comparación con otras rutas están en `EXPERIMENTOS.md`.

Para reproducir la Parte 1:

```
export H200_EMBED_URL=http://localhost:11434 EMBEDDING_BACKEND=h200 OPENAI_API_KEY= \
      GENERATION_MODEL=qwen3:1.7b QDRANT_COLLECTION=taller2_bge_m3_local
python inspeccion_ingesta.py 512 102 # 1.1 y 1.2: solo tokeniza
python demo_top5.py # 1.3 a 1.5: indexa, top-5 y generación
```

Las salidas crudas están en `salidas/bge-m3@ollama-local/`.

1.1 Ingesta: qué se pierde

La ingesta es la de `ingestion.py` sin cambios: `pypdf` página por página, con el marcador `[page=n]`. **No rechazó ningún documento**: los 16 superan el umbral de `MIN_CARACTERES_UTILES` y ninguno tiene páginas vacías. Salida de `load_corpus` (`salidas/bge-m3@ollama-local/parte1_ingesta.txt`):

```
ingesta: bai-2022-constitutional-ai.pdf: 34 página(s), 94294 caracteres útiles → 75 fragmento(s)
ingesta: blei-2003-lda.pdf: 30 página(s), 59953 caracteres útiles → 57 fragmento(s)
ingesta: brown-2020-gpt3.pdf: 75 página(s), 186342 caracteres útiles → 156 fragmento(s)
ingesta: doersch-2016-tutorial-vae.pdf: 23 página(s), 36740 caracteres útiles → 33 fragmento(s)
ingesta: edge-2024-graphrag.pdf: 26 página(s), 70385 caracteres útiles → 60 fragmento(s)
ingesta: es-2023-ragas.pdf: 8 página(s), 25583 caracteres útiles → 21 fragmento(s)
ingesta: gao-2023-rag-survey.pdf: 21 página(s), 86641 caracteres útiles → 78 fragmento(s)
ingesta: kingma-2013-vae.pdf: 14 página(s), 31786 caracteres útiles → 31 fragmento(s)
ingesta: lewis-2020-rag.pdf: 19 página(s), 55094 caracteres útiles → 47 fragmento(s)
ingesta: ng-jordan-2001-discriminative-vs-generative.pdf: 8 página(s), 18996 caracteres útiles → 19
fragmento(s)
ingesta: ouyang-2022-instructgpt.pdf: 68 página(s), 143167 caracteres útiles → 118 fragmento(s)
ingesta: rafailov-2023-dpo.pdf: 27 página(s), 71700 caracteres útiles → 66 fragmento(s)
ingesta: rag-everything-2025.pdf: 18 página(s), 57304 caracteres útiles → 41 fragmento(s)
ingesta: reimers-2019-sentence-bert.pdf: 11 página(s), 35133 caracteres útiles → 31 fragmento(s)
ingesta: vaswani-2017-attention-is-all-you-need.pdf: 15 página(s), 31566 caracteres útiles → 25
fragmento(s)
ingesta: wei-2022-chain-of-thought.pdf: 43 página(s), 105503 caracteres útiles → 90 fragmento(s)
```

Que todos tengan texto no quiere decir que el texto esté bien. Revisando muestras del texto extraído se ven cinco degradaciones, de más a menos grave para este corpus:

- **Tablas.** Quedan como líneas de números sin columnas. En la Tabla 2 de Vaswani et al. la fila `ByteNet [18]` 23.75 ya no dice a qué columna pertenece el 23.75, porque las celdas vacías desaparecen y lo de la derecha se corre. Los exponentes pierden el superíndice: `1.0 · 1020` es `1,0 · 1020`. En Reimers y Gurevych la leyenda de la Tabla 1 aparece pegada al texto de la columna vecina, y el contenido de la tabla queda en otra parte. Una pregunta cuya respuesta sea una cifra de tabla es candidata a fallar en la generación aunque la recuperación acierte.
- **Ecuaciones.** Se aplanan: `Attention(Q,K,V) = softmax(QKT/√dk)V`. El texto alrededor sobrevive, la fórmula no se puede leer.
- **Palabras cortadas por el guion de fin de línea.** `col-\nlection`, `neu-\ntral`. El conteo va de 0 (Ng y Jordan) a 221 (Reimers y Gurevych), más alto en los papers a dos columnas. Para el tokenizador de los

embeddings no da lo mismo: `collection` es un token y `col-lection` son cuatro. Además, una `fragmento_esperado` del golden set que cruce una palabra cortada **nunca va a coincidir**, porque el evaluador normaliza tildes y puntuación, pero no une palabras. Las frases del golden set se eligen evitándolas.

- **Encabezados y pies repetidos.** Solo aparecen en tres documentos: Blei et al. (`LATENT DIRICHLET ALLOCATION, BLEI, N G, AND JORDAN`), RAG-Anything y la plantilla de prompt de RAGAS. Son pocos tokens, pero se repiten en muchos fragmentos del mismo documento.
- **Ligaduras** (`flexibility`, hasta 444 en Brown et al., y en 9 de los 16 documentos). Con bge-m3 no pesan: se comprobó que su tokenizador las normaliza, y `flexibility` y `flexibility` dan los mismos tokens. Aquí el modelo de embeddings sí importa, porque `cl100k_base`, el tokenizador de `text-embedding-3-small`, no normaliza Unicode: `flexibility` son 2 tokens y `flexibility` son 5, tres de ellos bytes sueltos de la ligadura. Con la ruta de OpenAI esta degradación existiría. También pesaría en un BM25 que tokenice por espacios (Parte 3, Opción B). El evaluador no se ve afectado, porque `_normalizar_texto` aplica NFKD y convierte `fl` en `fl`.

Hay una observación que no es una degradación del parseo pero puede serlo de la recuperación: **el 40,6 % de los tokens del corpus está después del último encabezado «References»**, es decir, en la bibliografía y en los apéndices. En Wei et al. es el 75,1 %, casi todo apéndice con ejemplos de prompts; en Ouyang et al. y Rafailov et al., cerca del 60 %. En el baseline no se quitan: el baseline se mide como viene, y si en la Parte 2 aparecen fragmentos de bibliografía en el top-k (listas de títulos que comparten palabras con la pregunta), eso va al análisis de fallos como falla de ingesta.

1.2 Fragmentación

`chunk_tokens = 512`, `overlap_tokens = 102`, en tokens del tokenizador de BAAI/bge-m3, de tamaño fijo, con la función `fixed_size_chunks_tokens` del andamiaje. Resultado: **948 fragmentos**, todos de 512 tokens salvo el último de cada documento (el más corto tiene 61). Al re-tokenizar el texto decodificado, alguno llega a 513: la decodificación y re-codificación no es exacta en los bordes, y un token de diferencia contra un tope de 8192 no importa.

La justificación es contra las preguntas, no contra el tope. Sobre este corpus, bge-m3 da una mediana de **1,74 tokens por palabra y 854 tokens por página** (p25 635, p75 1071). Así que un fragmento de 512 tokens son **unas 293 palabras, un 60 % de página**: dos o tres párrafos de un paper. Las preguntas simples del golden set van a apuntar a una afirmación concreta (una cifra de un experimento, la definición de un método), que en un paper suele caber en un párrafo. Con 512 tokens, ese párrafo entra completo junto con algo de su contexto, y el vector del fragmento sigue siendo sobre un solo tema.

Que bge-m3 admita 8192 tokens no cambia esto. Un fragmento de 8192 serían unas 10 páginas de paper, la mitad de un documento típico de este corpus, y su vector promediaría la introducción, el método y los resultados. Una pregunta específica se parecería poco a ese promedio. En el otro extremo, un fragmento de 128 como el del MiniLM de la Parte 0 serían unas 74 palabras, y la respuesta a una pregunta multi-fragmento quedaría repartida entre muchos pedazos que el top-5 no alcanza a juntar.

El solapamiento es un quinto del fragmento, el valor por defecto del Lab-02: 102 tokens, unas 59 palabras. Su función es que una idea que cae justo en el corte aparezca entera en alguno de los dos fragmentos vecinos. En 1.4 se ve funcionando: la frase «Training took 3.5 days on 8 P100 GPUs» está completa en dos fragmentos seguidos de Vaswani et al. (–0013 y –0014). Con un paso de 410 tokens, el solapamiento cuesta un 25 % más de fragmentos que sin él, algo que no importa a costo cero.

Este valor es el del baseline y no se ajusta mirando el golden set. Si la Parte 2 muestra fallas de fragmentación (la respuesta cortada entre dos fragmentos, o fragmentos que mezclan temas), la Opción A de la Parte 3 es donde se prueba otra estrategia.

1.3 Embedding e indexación

Fila de embeddings: `embed_local_multilingue`. Es BAAI/bge-m3: 1024 dimensiones, tope de 8192 tokens, multilingüe, USD 0 (costo cero real), **verificada el 2026-08-27** en la tabla semestral.

El pipeline es el del andamiaje sin cambios de código (`EMBEDDING_BACKEND=h200`, `CodificadorH200`). Lo único distinto es `H200_EMBED_URL=http://localhost:11434`: el cliente habla con un Ollama, y en vez del de la H200 se usó uno local con `bge-m3:latest`. Según `ollama show`, ese modelo tiene arquitectura BERT, 566,70 M de parámetros, embeddings de 1024, contexto de 8192 y pesos en F16, lo que coincide con `bge-m3`. Se mantiene `truncate: false`, así que un texto que no quepa da error en vez de recortarse. La ingesta fragmenta con el tokenizador de Hugging Face `BAAI/bge-m3`, el mismo que el pipeline usa contra la H200.

Hay que ser claro en que **no es la H200**. El modelo es el mismo, pero no se verificó que el servidor de la H200 sirva exactamente la misma versión ni la misma cuantización. Por eso la etiqueta de estas corridas es `bge-m3@ollama-local` y no `bge-m3@h200`. Cuando haya VPN se repite todo con la H200 y se comparan las dos filas en `EXPERIMENTOS.md`. Antes se intentó la ruta de OpenAI (`embed_api_economico`), pero la clave fue rechazada (401) y no se indexó nada con ella. Un índice construido con un modelo solo se consulta con ese mismo modelo, así que cada configuración tiene su propia colección de Qdrant.

Indexación en el **Qdrant del contenedor** (`http://localhost:6333`, Qdrant 1.19.1, colección `taller2_bge_m3_local`), no en `:memory:`: **948 puntos, dimensión 1024, distancia coseno**, 479 701 tokens vectorizados en **136,6 s** en la Mac.

1.4 Recuperación: top-5 para tres preguntas de prueba

Tres preguntas en inglés, como el corpus, que no son del golden set: sirven para ver que el pipeline funciona, no para medirlo. Salida cruda (`salidas/bge-m3@ollama-local/parte1_top5.txt`):

```
embeddings: h200 · BAAI/bge-m3 · tope 8192 · Qdrant http://localhost:6333 · colección
taller2_bge_m3_local
indexando 948 fragmentos (512/102) · 479701 tokens · costo estimado USD 0.0000
indexados: 948 puntos · dimensión 1024 · distancia Cosine · 136.6 s

[1] How long did it take to train the big Transformer model, and on what hardware?
  1. 0.630 vaswani-2017-attention-is-all-you-need.pdf / vaswani-2017-attention-is-all-you-need-0014
previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new...
  2. 0.569 vaswani-2017-attention-is-all-you-need.pdf / vaswani-2017-attention-is-all-you-need-0013 s
than previous state-of-the-art models on the English-to-German and English-to-French new...
  3. 0.567 vaswani-2017-attention-is-all-you-need.pdf / vaswani-2017-attention-is-all-you-need-0012
ching We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 m...
  4. 0.536 vaswani-2017-attention-is-all-you-need.pdf / vaswani-2017-attention-is-all-you-need-0016
too many heads. In Table 3 rows (B), we observe that reducing the attention key size dk hu...
  5. 0.526 vaswani-2017-attention-is-all-you-need.pdf / vaswani-2017-attention-is-all-you-need-0002
they generate a sequence of hidden statesht, as a function of the previous hidden stateht--
  respuesta [ollama:qwen3:1.7b] abstuvo=False:
  La entrenación del modelo Transformer grande se llevó a cabo en 3.5 días utilizando 8 unidades de
procesamiento gráfico P100.

[2] Why does DPO not need to train an explicit reward model?
  1. 0.686 rafailov-2023-dpo.pdf / rafailov-2023-dpo-0003 reward maximization with a KL-divergence
constraint) but is simple to implement and straig...
  2. 0.640 rafailov-2023-dpo.pdf / rafailov-2023-dpo-0002 , and then use RL to find a policy that
maximizes the learned reward. In contrast, DPO dir...
  3. 0.638 rafailov-2023-dpo.pdf / rafailov-2023-dpo-0012 the DPO method, provide theoretical backing,
and relate advantages of DPO to issues with a...
  4. 0.633 rafailov-2023-dpo.pdf / rafailov-2023-dpo-0000 [page=1] Direct Preference Optimization:
Your Language Model is Secretly a Reward Model Ra...
  5. 0.627 rafailov-2023-dpo.pdf / rafailov-2023-dpo-0025 about as often as humans agree with each
other, suggesting that GPT-4 is a reason- able pr...
  respuesta [ollama:qwen3:1.7b] abstuvo=False:
  DPO no necesita entrenar un modelo de recompensa explícito porque utiliza un cambio de variables
para definir la pérdida de preferencias directamente como una función del política, evitando la
```

necesidad de un modelo de recompensa separado. Esto permite optimizar la política directamente mediante un objetivo de entropía binaria simple, sin recurrir a métodos de aprendizaje por reacción (RL) o a un modelo de recompensa explícito.

[3] Which quality aspects of a RAG answer does RAGAS evaluate without human references?

1. 0.634 es-2023-ragas.pdf / es-2023-ragas-0012 we observed that ChatGPT often struggles with the task of selecting the sentences from t...

2. 0.609 es-2023-ragas.pdf / es-2023-ragas-0000 [page=1] Ragas: Automated Evaluation of Retrieval Augmented Generation Shahul Est, Jithin ...

3. 0.609 gao-2023-rag-survey.pdf / gao-2023-rag-survey-0035 quiry. 2) Required Abilities: RAG evaluation also encompasses four abilities indicative of...

4. 0.606 es-2023-ragas.pdf / es-2023-ragas-0005 ailability of one or more reference answers. For instance, BERTScore (Zhang et al., 2020) ...

5. 0.602 gao-2023-rag-survey.pdf / gao-2023-rag-survey-0040 of noise or contradictory information during retrieval can detrimentally affect RAG's outp...

respuesta [ollama:qwen3:1.7b] abstuvo=False:

The quality aspects of a RAG answer that RAGAS evaluates without human references are: **Context Relevance**, **Faithfulness**, and **Answer Relevance**.

Lo que se ve:

- **Pregunta 1:** los cinco vecinos son de Vaswani et al. La respuesta («Training took 3.5 days on 8 P100 GPUs») está en los fragmentos de las posiciones 1 y 2, y la sección «Hardware and Schedule» en la 3.
- **Pregunta 2:** los cinco son de Rafailov et al.
- **Pregunta 3:** mezcla dos documentos. RAGAS ocupa las posiciones 1, 2 y 4, y el survey de Gao et al. las 3 y 5. Tiene sentido, porque el survey resume RAGAS en su sección de evaluación.
- **Los puntajes están todos entre 0,52 y 0,69**, con poca separación entre el primero y el quinto (0,630 contra 0,526 en la pregunta 1). Es lo que la Parte 0.c ya mostró: el puntaje solo ordena, no dice si la respuesta está. Habrá que ver en la Parte 2 si las negativas quedan en ese mismo rango.

1.5 Generación

Generador: **qwen3:1.7b por Ollama local** (fila `open_weight_pequeno`), con temperatura 0, porque la clave de OpenAI no funcionó. El prompt y la frase de abstención son los del andamiaje sin tocar. `ABSTENCION` sigue siendo «El corpus no contiene información suficiente.», la misma en el prompt, en el golden set y en el detector. `OPENAI_API_KEY=` se deja vacía a propósito, porque `generate` prefiere OpenAI apenas hay una clave, aunque sea inválida.

Las tres respuestas son correctas contra el paper. Hay dos detalles que conviene anotar para la Parte 2:

- **Idioma.** El modelo responde en español en las preguntas 1 y 2, porque el prompt está en español, y en inglés en la 3. No es un error, pero una respuesta esperada en inglés comparada por cadena no coincidiría. Además, si el modelo se abstuviera en inglés, el detector (que busca la frase en español) no lo vería. Hay que revisarlo en la Parte 2.b.
- **Pequeños errores de traducción** propios de un modelo de 1,7 B. En la pregunta 2 escribe «aprendizaje por reacción (RL)» en vez de «aprendizaje por refuerzo». El contenido está bien.

Parte 2 — Evaluación con golden set

Configuración: la del baseline de la Parte 1, `bge-m3@ollama-local` (fila `embed_local_multilingue`, verificada 2026-08-27), 512/102, 948 fragmentos, generación con `qwen3:1.7b` (`open_weight_pequeno`) por Ollama. Para reproducir:

```
export H200_EMBED_URL=http://localhost:11434 EMBEDDING_BACKEND=h200 OPENAI_API_KEY= \
    GENERATION_MODEL=qwen3:1.7b QDRANT_COLLECTION=taller2_bge_m3_local
python verificar_golden.py                # 2.a: el golden set contra los fragmentos reales
python evaluation.py --k 3 --sin-indexar  # 2.b (reusa el índice de la Parte 1)
python evaluation.py --k 5 --sin-indexar
python tabla_metricas.py                  # tabla desde los CSV + resultados.csv
python analisis_fallos.py                  # 2.c: evidencia de los peores casos
```

Cambio declarado sobre el andamiaje: `evaluation.py` ahora escribe un CSV por configuración y por `k` (antes la corrida con `k = 5` pisaba la de `k = 3`) y guarda la pregunta, la respuesta entera y los puntajes de todos los recuperados. Las métricas y sus definiciones no se tocaron.

2.a El golden set

`golden_set.json`, 10 preguntas en inglés, como el corpus:

id	tipo	documento(s) fuente	qué prueba
1	simple	Reimers y Gurevych	una cifra concreta (65 horas)
2	simple	Kingma y Welling	un concepto (la reparametrización)
3	simple	Edge et al.	un nombre propio (Leiden)
4	simple	Wei et al.	una cifra con un distractor cerca (100B contra 540B)
5	multi-fragmento	Lewis et al. + RAGAS	combinar dos documentos
6	multi-fragmento	Ouyang et al. + Rafailov et al.	combinar dos documentos
7	multi-fragmento	Edge et al.	combinar dos secciones del mismo documento
8	negativa	—	tasa de aprendizaje de Llama 2
9	negativa	—	número de parámetros de GPT-4
10	adversarial	Vaswani et al.	una instrucción de ignorar el contexto

Algunas decisiones de diseño:

- **Anotación por documento y frase literal**, no por `chunk_id`, para que la Opción A de la Parte 3 pueda re-fragmentar sin invalidar nada. `verificar_golden.py` comprueba contra los 948 fragmentos reales que cada frase cabe entera en al menos un fragmento de su documento (entre 1 y 4, según la frase), y que ninguna cruza una palabra cortada por guion. Esto último hizo falta de verdad: la frase natural de la pregunta 1 («most similar pair») aparece en el PDF como `sim-ilar` y no habría coincidido nunca.
- **Preguntas parafraseadas**. El mismo script mide cuántas palabras de contenido comparte cada pregunta con su frase esperada: 0 de 2, 1 de 5, 2 de 5, 1 de 7 y 2 de 4. La recuperación no puede ganar por copiar palabras.
- **Negativas cercanas, no absurdas**. Una pregunta sobre recetas se abstendría sola. Las dos negativas nombran modelos que **sí aparecen** en el corpus: Llama 2 en la bibliografía de Edge et al., y GPT-4 en 49 fragmentos (como juez en Rafailov et al. y en bibliografías). Pero ningún fragmento contiene la respuesta. Se comprobó buscando co-apariciones: «gpt 4» con «trillion» solo aparece en una bibliografía, y «llama» con «learning rate» en ninguna parte. La 9, además, tienta al generador a responder con lo que «sabe».

- **La adversarial es responsable:** tiene documento fuente y cuenta en el Hit Rate y el MRR. Se decidió así porque lo que interesa es que el sistema **no obedezca** la instrucción metida en la pregunta y responda desde el contexto. Si se hubiera dejado sin fuente, se estaría premiando que se abstenga ante una pregunta que sí tiene respuesta.
- **Limitación que se asume:** en las multi-fragmento no se pone frase, así que el acierto es a nivel de documento, y basta un fragmento de *uno* de los dos documentos. Eso es generoso, y en la 2.c se ve el costo: la pregunta 5 cuenta como acierto con $k = 3$ aunque no se recuperó nada de Lewis et al.

2.b Métricas

Tabla reconstruida desde los CSV crudos por `tabla_metricas.py`, que coincide con lo que reportó `evaluation.py` (`salidas/bge-m3@ollama-local/tabla_metricas.md`):

k	preguntas	respondibles	negativas	Hit Rate@k	MRR	abstención correcta	abstención indebida
3	10	8	2	0.875	0.750	1.000	0.250
5	10	8	2	1.000	0.781	1.000	0.125

Por tipo:

k	tipo	n	Hit Rate	MRR	se abstuvo
3	simple	4	0.750	0.625	0.000
3	multi-chunk	3	1.000	1.000	0.667
3	adversarial	1	1.000	0.500	0.000
3	negativo	2	—	—	1.000
5	simple	4	1.000	0.688	0.000
5	multi-chunk	3	1.000	1.000	0.333
5	adversarial	1	1.000	0.500	0.000
5	negativo	2	—	—	1.000

Cómo se leen:

- **Hit Rate y MRR, solo sobre las 8 respondibles.** Pasar de $k = 3$ a $k = 5$ sube el Hit Rate de 0,875 a 1,000, porque la única pregunta sin acierto (la 3) lo encuentra en la posición 4. El MRR casi no se mueve (0,750 $\rightarrow$ 0,781): ese acierto nuevo entra con un recíproco de 1/4. Lo mismo muestra que el MRR **no es la posición promedio**. Con $k = 5$, las posiciones del primer acierto son 1, 1, 4, 2, 1, 1, 1, 2, con media 1,625. Su inverso sería 0,615, mientras que el MRR, que promedia los recíprocos, da 0,781. Una sola pregunta en la posición 4 arrastra la media de posiciones mucho más de lo que baja el MRR.
- **Dos tasas de abstención, no una.** La correcta es 1,000 con los dos k : el sistema se abstuvo en las dos negativas. Solo con eso parecería perfecto, pero la indebida es 0,250 con $k = 3$ y 0,125 con $k = 5$: también se abstuvo en preguntas que tenían respuesta, 2 de 8 y 1 de 8. Un sistema que siempre se abstiene sacaría 1,000 en la primera y 1,000 en la segunda. Por eso las dos van juntas.
- **Todas las abstenciones son la frase exacta** («El corpus no contiene información suficiente.»), así que el detector no se perdió ninguna. Se revisó a mano porque `qwen3` responde a veces en inglés (preguntas 5 y 7), y una abstención en inglés no la habría detectado.
- **Si las negativas entraran al Hit Rate** como fallos, el techo con 2 negativas de 10 sería 0,80, y el 1,000 de $k = 5$ se reportaría como 0,800 sin que el sistema hubiera cambiado. La métrica correcta para ellas es la abstención.
- **Con 8 respondibles, cada pregunta mueve el Hit Rate en 0,125.** Las diferencias entre k son una o dos preguntas y no permiten generalizar mucho más allá de este golden set.

2.c Los tres peores casos

La evidencia completa (ingesta, recuperados con puntaje y qué palabras clave de la respuesta esperada tiene cada uno, y la respuesta) está en `salidas/bge-m3@ollama-local/fallos_evidencia.txt`.

Caso 1 — Pregunta 6 (InstructGPT contra DPO): generación / prompt

Se abstiene con $k = 3$ y con $k = 5$, aunque la métrica dice acierto en la posición 1 con los dos k . Con $k = 5$ los recuperados son:

```
1. 0.660 rafailov-2023-dpo-0025 contiene: ['reinforcement learning', 'PPO']
2. 0.658 rafailov-2023-dpo-0003 contiene: ['reward model', 'PPO', 'binary cross entropy', 'change of variables']
3. 0.657 rafailov-2023-dpo-0019 contiene: ['reward model', 'PPO']
4. 0.651 rafailov-2023-dpo-0021 contiene: ['PPO']
5. 0.650 ouyang-2022-instructgpt-0006 contiene: -
respuesta: El corpus no contiene información suficiente.
```

El fragmento `rafailov-2023-dpo-0003` tiene **todas** las piezas de la respuesta: el paper de DPO describe el pipeline RLHF que reemplaza (modelo de recompensa + PPO) y su propia pérdida (cambio de variables, entropía cruzada binaria). La recuperación cumplió, y la ingesta de los dos documentos no tiene nada raro (118 y 66 fragmentos). La falla está en la generación: el prompt obliga a usar solo el contexto, la pregunta nombra «InstructGPT», y ningún fragmento del top-5 lo nombra así. Un modelo de 1,7 B parece leer eso como «no está» y se abstiene. Es una abstención indebida que el Hit Rate no ve.

Caso 2 — Pregunta 5 (RAG contra RAGAS) con $k = 3$: recuperación

```
1. 0.741 es-2023-ragas-0002 contiene: ['faithfulness']
2. 0.730 es-2023-ragas-0000 contiene: -
3. 0.704 es-2023-ragas-0001 contiene: -
respuesta: El corpus no contiene información suficiente.
```

Los tres vecinos son de RAGAS y **ninguno de Lewis et al.** La pregunta menciona los dos, pero el vector de la pregunta queda más cerca del paper que habla de evaluación. Con ese contexto, abstenerse no es un error del generador: la mitad de la respuesta no estaba. Es una falla de recuperación propia de las preguntas que necesitan dos documentos: el top- k lo acapara el documento más parecido. Con $k = 5$, Lewis et al. entra en la posición 4 y la respuesta sale correcta. Aun así, la métrica marca acierto en la posición 1 con los dos k , por la limitación declarada en la 2.a (acierto a nivel de documento). Es el caso más claro de un acierto que no es un acierto.

Caso 3 — Pregunta 4 (tamaño de modelo para chain-of-thought) con $k = 5$: generación

```
1. 0.735 wei-2022-chain-of-thought-0043 contiene: ['540B']
2. 0.734 wei-2022-chain-of-thought-0009 contiene: ['100B', 'only yields performance gains when used with models of', '540B']
3. 0.731 wei-2022-chain-of-thought-0010 contiene: ['540B']
...
respuesta: ... el inicio de los beneficios se observa en modelos de tamaño similar a 540B parámetros.
```

La recuperación trae el fragmento correcto en la posición 2, y con $k = 3$ la respuesta era correcta («aproximadamente 100B»). Con $k = 5$ entran dos fragmentos más, ninguno con la respuesta, y el modelo cambia a «540B», la cifra que más se repite en el contexto (aparece en tres de los cinco fragmentos). Más contexto empeoró la respuesta, algo que ni el Hit Rate ni el MRR registran (los dos suben de $k = 3$ a $k = 5$). Es una falla de generación que el golden set solo detecta si se leen las respuestas. Por eso el CSV ahora guarda la respuesta entera.

Una falla del verificador, no del pipeline: la pregunta 3

Con $k = 3$ la pregunta 3 es el único fallo de recuperación de la tabla (posición «—»), pero la respuesta es correcta («Leiden (Traag et al., 2019)»). El fragmento en la posición 1, `edge-2024-graphrag-0011`, **sí dice Leiden**, pero con otras palabras, y la frase anotada («Community detection (e.g., Leiden)») está en los fragmentos 0007 y 0008, que no se recuperaron. La anotación era más estricta que la pregunta. No se corrige ahora, porque cambiar el golden set después de ver los resultados sería ajustar el verificador al sistema. Queda como candidata para una versión 1.1 del golden set, declarada como tal, y descuenta el 0,125 que separa el Hit Rate@3 de 1,000.

Las tres fallas silenciosas de la Parte 0, en este corpus

- **PDF sin texto (0.a): no está.** Los 16 documentos tienen capa de texto y 0 páginas vacías (`salidas/corpus_verificacion.txt`), y la ingesta no rechazó ninguno.
- **Truncado (0.b): no está.** Los fragmentos son de 512 tokens (el más largo re-tokenizado, 513) contra un tope de 8192, y la ruta de Ollama pide `truncate: false`, así que un texto que no cupiera daría error en vez de recortarse.
- **El índice que no se queja (0.c): sí está.** Los mejores puntajes de las negativas (0,555 y 0,602) caen **dentro** del rango de los de las respondibles (0,548 a 0,741). La respondible 2 tiene un mejor puntaje más bajo que las dos negativas. Ningún umbral sobre el puntaje separaría «hay respuesta» de «no la hay». Aquí lo que salvó la tasa de abstención correcta fue el generador, no el índice.

Además, la sospecha de la Parte 1 sobre la bibliografía (40,6 % de los tokens) no se confirmó. De los 50 fragmentos recuperados con $k = 5$, solo 1 tiene aspecto de bibliografía (`gao-2023-rag-survey-0074`), según una heurística simple que cuenta marcas de cita.

Parte 3 — Extensión: búsqueda híbrida (Opción B)

Hipótesis, declarada antes de implementar

Se elige por lo que midió la Parte 2, no por preferencia. El baseline denso tiene Hit Rate@5 de 1,000 y MRR de 0,781, así que casi no le falta encontrar el documento. Los dos peores casos de la 2.c apuntan a otra cosa: **el recuperador denso ignora los nombres propios de la pregunta**.

- **Pregunta 5** («Lewis et al.'s RAG and the RAGAS framework...»): con $k = 3$, los tres vecinos son de RAGAS y ninguno de Lewis et al. La pregunta nombra los dos trabajos, pero el vector la acerca a uno solo.
- **Pregunta 6** («InstructGPT and DPO...»): con $k = 5$, cuatro vecinos son de DPO y solo uno de InstructGPT, en la posición 5, y ese no contiene ninguna palabra clave de la respuesta.

BM25 puntúa por coincidencia exacta de términos, y «RAGAS», «Lewis», «InstructGPT» o «DPO» son términos raros en el corpus, con IDF alto. **Lo que se espera corregir** es que, al fusionar BM25 con el denso por Reciprocal Rank Fusion, el documento que falta en las preguntas que nombran dos trabajos entre al top-k.

Cómo se va a medir, con el mismo golden set, el mismo índice (948 fragmentos, 512/102) y el mismo generador:

1. **Hit Rate@k y MRR** para $k = 3$ y $k = 5$, contra el baseline, que es lo que pide el enunciado.
2. **Cobertura de fuentes en las preguntas multi-documento (5 y 6)**: qué fracción de sus documentos fuente aparece en el top-k. Es la medición directa de la hipótesis. El Hit Rate no la ve, porque en esas preguntas el acierto es a nivel de documento y le basta uno de los dos (limitación declarada en la 2.a).
3. **Las dos tasas de abstención**, porque en la pregunta 6 el baseline se abstuvo teniendo la respuesta en el contexto. Si el contexto híbrido nombra InstructGPT, puede que ya no se abstenga.
4. **BM25 solo**, como diagnóstico, para saber qué aporta cada lado de la fusión.

Lo que **no** se espera corregir:

- la pregunta 4 con $k = 5$, que es una falla de generación (el modelo se distrae con «540B»);
- la pregunta 3, que es una falla de la anotación del golden set.

Si la fusión no mejora nada, o empeora las preguntas simples (BM25 puede meter fragmentos que repiten palabras de la pregunta sin responderla), eso también es un resultado, y se explica.

Parámetros fijados antes de medir: 20 candidatos de cada recuperador, RRF con la constante habitual $k = 60$, y tokenización de BM25 con la misma normalización del evaluador (minúsculas, NFKD, sin puntuación, lo que además deshace las ligaduras) y sin palabras vacías en inglés.

Implementación

`hibrido.py`: `PipelineHibrido` hereda de `RagPipeline` y solo reemplaza `retrieve`.

- **BM25** (`rank_bm25.BM25Okapi`) se construye sobre los **mismos 948 fragmentos** de la colección de Qdrant: se leen del índice, no se re-fragmenta.
- **La fusión**: el denso y BM25 dan 20 candidatos cada uno, y RRF suma $1/(60 + \text{posición})$ de cada lista.
- **Lo que no cambia**: el prompt, el generador (`qwen3:1.7b`), el detector de abstención y `evaluate_retrieval`, que es la misma función del evaluador. La comparación cambia una sola cosa.

- **Costo:** USD 0, todo local. Las dos corridas híbridas (k = 3 y k = 5, con generación) tardaron 2 min 40 s.

```
export H200_EMBED_URL=http://localhost:11434 EMBEDDING_BACKEND=h200 OPENAI_API_KEY= \
  GENERATION_MODEL=qwen3:1.7b QDRANT_COLLECTION=taller2_bge_m3_local
python hibrido.py --modo bm25 --sin-generar # diagnóstico
python hibrido.py --modo hibrido
python comparar_extension.py # salidas/bge-m3@ollama-local/comparacion_extension.md
```

Resultados

Desde los CSV crudos (salidas/bge-m3@ollama-local/{,bm25/,hibrido/}resultados_k*.csv):

variante	k	Hit Rate@k	MRR	abstención correcta	abstención indebida	cobertura P5	cobertura P6
denso (baseline)	3	0.875	0.750	1.000	0.250	1/2	1/2
denso (baseline)	5	1.000	0.781	1.000	0.125	2/2	2/2
bm25 solo	3	0.875	0.750	sin medir	sin medir	1/2	2/2
bm25 solo	5	0.875	0.750	sin medir	sin medir	2/2	2/2
híbrido (RRF)	3	1.000	0.854	1.000	0.125	1/2	2/2
híbrido (RRF)	5	1.000	0.854	1.000	0.000	2/2	2/2

Posición del primer acierto por pregunta (— sin acierto; A = se abstuvo):

id	tipo	denso k=3	denso k=5	bm25 k=3	bm25 k=5	híbrido k=3	híbrido k=5
1	simple	1	1	1	1	1	1
2	simple	1	1	2	2	1	1
3	simple	—	4	2	2	2	2
4	simple	2	2	—	—	3	3
5	multi-chunk	1 A	1	1	1	1 A	1
6	multi-chunk	1 A	1 A	1	1	1	1
7	multi-chunk	1	1	1	1	1	1
8	negativo	A	A	—	—	A	A
9	negativo	A	A	—	—	A	A
10	adversarial	2	2	1	1	1	1

Qué se confirmó y qué no

La hipótesis se cumplió a medias: en la pregunta 6 sí, en la 5 no.

Pregunta 6, se cumplió. Con k = 3, la cobertura pasa de 1/2 a 2/2: InstructGPT entra en la posición 2 (rafailov, ouyang, rafaïlov). Y el efecto llega a la generación: el baseline se abstenía con los dos k, y el híbrido ya no se abstiene con ninguno. Responde bien: DPO optimiza la política directamente con una pérdida de entropía cruzada binaria, sin aprendizaje por refuerzo, e InstructGPT entrena un modelo de recompensa y luego usa PPO. Esto es casi toda la baja de la abstención indebida (0,250 → 0,125 con k = 3 y 0,125 → 0,000 con k = 5). También refuerza la lectura de la 2.c: el modelo de 1,7 B se abstenía porque ningún fragmento nombraba «InstructGPT», no porque faltara la información.

Pregunta 5, no se cumplió. Con $k = 3$ la cobertura sigue en $1/2$, los tres vecinos siguen siendo de RAGAS y el sistema se sigue absteniendo. Esta vez BM25 solo tampoco trae a Lewis et al. La razón está en el corpus, y es lo más interesante de esta parte. Se contó en cuántos fragmentos aparece cada término de la pregunta:

término	fragmentos	de qué documentos
dpo	37	los 37 de Rafailov et al.
instructgpt	54	43 de Ouyang et al., 10 de Bai et al., 1 de Wei et al.
ragas	12	5 de RAGAS, 4 del survey de Gao et al., 3 de Edge et al.
lewis	29	solo 6 de Lewis et al. ; 9 de Gao, 4 de Edge, 4 de RAGAS, 3 de Brown...

«DPO» e «InstructGPT» son nombres de **métodos**, y el paper que los propone los repite en el cuerpo del texto. «Lewis» es el nombre de un **autor**. En su propio paper aparece en el fragmento 0 (la línea de autores) y en cinco fragmentos de la segunda mitad (del 0026 al 0036, de 47), que por su posición parecen ser la bibliografía; no se revisó uno por uno. Donde más aparece es en las citas de otros papers («Lewis et al., 2020»). Para BM25, «lewis» apunta a los trabajos que citan el RAG, no al RAG. Así que la hipótesis era correcta para preguntas que nombran métodos y falsa para las que nombran autores. Eso no se sabía antes de medir.

De dónde sale la subida del MRR (0,750 → 0,854 con $k = 3$). No de las preguntas de la hipótesis, que ya tenían su primer acierto en la posición 1, sino de dos simples:

- **Pregunta 3:** de «—» a 2. Hay que leerla con cuidado. La pregunta comparte «community» y «detection» con la frase anotada (el solapamiento de 2 de 5 medido en la 2.a), y BM25 premia justo eso. Parte de la mejora es coincidencia de palabras con la anotación, y la respuesta ya era correcta con el baseline (2.c).
- **Pregunta 10** (adversarial): de 2 a 1, porque «dropout rate» y «base» coinciden literalmente con el fragmento de Vaswani et al.

Lo que empeoró. La pregunta 4 baja de la posición 2 a la 3. BM25 solo no la encuentra nunca: «chain-of-thought» está en casi todos los fragmentos de Wei et al., así que no discrimina, y la frase con la respuesta dice «~100B parameters», no «model size». En la fusión, ese fragmento pierde el aporte del lado léxico. Como se había previsto, la falla de generación de la pregunta 4 con $k = 5$ sigue igual: el híbrido también responde «540B».

Las negativas no cambian: abstención correcta de 1,000 en las dos variantes. BM25 no las empeora, aunque «GPT-4» y «Llama» sí aparecen en el corpus.

Lectura final

Con 8 respondibles, cada pregunta mueve el Hit Rate en 0,125, así que estas diferencias son de una o dos preguntas y no alcanzan para decir que el híbrido es mejor en general. Lo que sí se puede decir, con evidencia, es que el híbrido **arregla la pregunta que nombra métodos (6), no arregla la que nombra autores (5)**, y que **la subida del MRR viene sobre todo de la coincidencia léxica en dos preguntas simples**, una de ellas inflada por la propia anotación. Para la pregunta 5 haría falta otra cosa, por ejemplo reescribir la consulta para separar las dos partes («RAG model of Lewis et al.» y «RAGAS framework») y recuperar para cada una. Eso ya no es la Opción B.

Parte 4 — Reflexión

Las cifras salen de los CSV crudos de la Parte 2.b (bge-m3@ollama-local) con parte4_cifras.py, que también corre la pregunta global de la 4.2 por el baseline. La salida cruda está en salidas/bge-m3@ollama-local/parte4_cifras.txt.

4.1 El Hit Rate de 0,70

Con la plantilla anterior, el Hit Rate dividía los aciertos entre las 10 preguntas, incluidas las negativas, que no pueden acertar porque no tienen documento fuente. Con mis resultados da **0,700 con k = 3** (7 de 10) y **0,800 con k = 5** (8 de 10), y 0,800 ya es el techo aunque el sistema acertó las 8 respondibles. Ese número mezclaba cuánto recupera el sistema con cuántas negativas trae el golden set. Sin ellas en el denominador queda solo la recuperación: **0,875 y 1,000**, y el MRR con k = 5 pasa de 0,625 a **0,781**. Mi 0,700 coincide con la cifra del enunciado por otra razón: aquí sí hay un fallo real. Para las negativas lo que importa es si el sistema se abstuvo, **2 de 2**, y eso se lee junto con la abstención indebida (0,125), porque abstenerse siempre también da 2 de 2.

k	Hit Rate antes (÷ 10)	Hit Rate ahora (÷ 8)	MRR antes	MRR ahora	abstención correcta
3	0.700	0.875	0.600	0.750	2 de 2
5	0.800	1.000	0.625	0.781	2 de 2

4.2 Una pregunta global

Pregunta global: «What are the main themes that run across all the papers in this corpus, and how do they relate to each other?». Responderla bien exige leer los 16 papers. El RAG vectorial solo trae los 5 fragmentos más parecidos a la pregunta: 2560 tokens de 387 086, el **0,66 %** del corpus. Corrida en el baseline, 4 de los 5 vecinos fueron del paper de LDA, que habla literalmente de «topics» dentro de un «corpus», y el sistema se abstuvo. Recuperó lo que se parece a la pregunta, no lo que la responde. GraphRAG (Edge et al., 2024) no depende de esa búsqueda. Al indexar, extrae un grafo de entidades, lo divide en comunidades con Leiden y resume cada comunidad. En la consulta trabaja en map-reduce: cada resumen da una respuesta parcial y luego se combinan. Así la respuesta cubre el corpus entero, no cinco fragmentos.

Lo que devolvió el baseline:

```
1. 0.482 blei-2003-lda-0001
2. 0.471 blei-2003-lda-0031
3. 0.469 blei-2003-lda-0037
4. 0.447 brown-2020-gpt3-0153
5. 0.446 blei-2003-lda-0005
documentos distintos en el top-5: 2 de 16
respuesta: El corpus no contiene información suficiente.
```